



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251187
Nama Lengkap	Benedictina Andhika Chinor Jodie Soesila
Minggu ke / Materi	05 / Modular Programming

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Fungsi, Argument dan Parameter

```
nama = input("Masukkan nama: ")
print("Hallo ", nama, " selamat pagi!")
```

Program ini meminta pengguna untuk memasukkan sebuah nama, lalu program akan memberikan sapaan sesuai dengan nama yang dimasukkan. Dalam program ini digunakan dua fungsi bawaan Python, yaitu **input()** dan **print()**. Fungsi **input()** berfungsi untuk menerima masukan dari pengguna, sedangkan **print()** digunakan untuk menampilkan teks ke layar.

Secara umum, fungsi adalah sekumpulan instruksi yang digabungkan untuk mencapai tujuan tertentu, memiliki kegunaan khusus, dan dapat dipakai kembali. Hubungannya dengan *modular programming* yaitu fungsi membantu membagi program besar menjadi bagian-bagian kecil (modul) yang lebih terstruktur, sehingga setiap bagian memiliki peran spesifik dan bisa digunakan ulang.

Berdasarkan asalnya, fungsi terbagi menjadi dua jenis:

- **Fungsi bawaan (built-in function)**, yaitu fungsi yang sudah tersedia dalam Python. Daftar lengkapnya dapat dilihat di dokumentasi resmi Python.
- **Fungsi buatan (user-defined function)**, yaitu fungsi yang dibuat sendiri oleh programmer sesuai kebutuhan program.

Sebagai contoh penerapan, berikut ditunjukkan fungsi **tambah()** yang digunakan menghitung penjumlahan dari dua bilangan yang dimasukkan:

```
def tambah(a, b):
    hasil = a + b
    return hasil
```

- **def** sendiri digunakan untuk mendefinisikan atau membuat sebuah fungsi
- Nama fungsi yang dibuat adalah **tambah()**.

- Isi fungsi harus ditulis dengan indentasi (menjorok ke dalam satu tab) sebagai penanda blok kode, yang terlihat pada bagian **tambah(a, b)**:
- Fungsi **tambah()** membutuhkan dua argumen yang akan dikenali sebagai parameter **a** dan **b**.
- Fungsi tersebut menghasilkan nilai hasil penjumlahan yang dapat disimpan dalam suatu variabel.
- **return** digunakan untuk mengembalikan atau menghasilkan nilai dari suatu fungsi.

Kegunaan fungsi tersebut dapat dilihat pada kode program berikut ini:

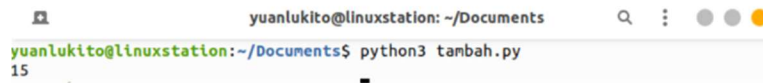
```
def tambah(a, b):
    hasil = a + b
    return hasil

c = tambah(10, 5)
print(c)
```

Jalannya program tersebut adalah sebagai berikut:

- Baris 1 merupakan komentar sehingga tidak dijalankan oleh interpreter Python.
- Baris 2 sampai baris 4 berisi pendefinisian fungsi **tambah()**, namun bagian ini belum dijalankan sampai fungsi tersebut dipanggil.
- Baris 5 dan baris 6 merupakan baris kosong serta komentar sehingga diabaikan oleh Python.
- Pada baris 7, variabel **c** diisi dengan nilai hasil dari pemanggilan fungsi **tambah()** dengan argumen 10 dan 5.
- Setelah fungsi dipanggil, program berpindah ke baris 2, di mana parameter **a** dan **b** menerima nilai **a = 10** dan **b = 5**.
- Pada baris 3 dilakukan proses penjumlahan sehingga variabel **hasil** berisi nilai **10 + 5 = 15**.
- Pada baris 4 digunakan keyword **return** untuk mengembalikan nilai **hasil**, yaitu 15, yang menandakan fungsi telah selesai dijalankan.

- Setelah itu program kembali ke baris 7 dan nilai yang dikembalikan fungsi disimpan ke dalam variabel **c**.
- Pada baris 8 nilai **c** ditampilkan ke layar sehingga menghasilkan output **15**.



```
yuanlukito@linuxstation: ~/Documents
yuanlukito@linuxstation:~/Documents$ python3 tambah.py
15
```

Gambar 1.1 Hasil dari pemanggilan fungsi tambah(). Sumber: <https://sl1nk.com/MDLRL>

Return Value

Secara umum, berdasarkan hasil yang dihasilkan oleh suatu fungsi, terdapat dua jenis fungsi, yaitu fungsi yang tidak mengembalikan nilai dan fungsi yang mengembalikan nilai. Fungsi yang tidak mengembalikan nilai biasanya disebut sebagai **void function**. Sebagai contoh, fungsi **print_twice()** merupakan salah satu fungsi yang tidak mengembalikan nilai.

```
def print_twice(message):
    print(message)
    print(message)

print_twice("Hello World!")
```

Fungsi **print_twice()** memerlukan satu parameter yaitu **message**. Fungsi ini akan menampilkan isi dari variabel **message** sebanyak dua kali. Namun, fungsi **print_twice()** tidak menghasilkan nilai yang dapat digunakan pada proses selanjutnya. Secara teknis, fungsi tersebut sebenarnya tetap mengembalikan suatu nilai, yaitu **None**, jika fungsi dipanggil seperti pada contoh berikut.



Gambar 1.2 Fungsi tambah() belum dikenali karena didefinisikan di bawahnya. Sumber:

<https://sl1nk.com/MDLRL>

```
def print_twice(message):  
    print(message)  
    print(message)  
print(print_twice("Hello World!"))
```

Output yang dihasilkan adalah:

```
Hello World!  
Hello World!  
None
```

Berbeda halnya dengan fungsi **tambah()** berikut ini:

```
def tambah(a, b, c):  
    hasil = a + b + c  
    return hasil
```

Fungsi **tambah()** memerlukan tiga parameter, yaitu **a**, **b**, dan **c**. Di dalam fungsi tersebut terdapat variabel **hasil** yang berisi nilai penjumlahan dari ketiga parameter tersebut, yaitu **a + b + c**. Nilai yang tersimpan dalam variabel **hasil** kemudian dikembalikan sebagai keluaran dari fungsi menggunakan keyword **return**. Syntax **return** digunakan untuk mengembalikan nilai sebagai hasil dari suatu fungsi sekaligus menandakan bahwa proses dalam fungsi tersebut telah selesai dijalankan.

```
def tambah(a, b, c):  
    hasil = a + b + c  
    return hasil
```

```
nilai1 = 70  
nilai2 = 85  
nilai3 = 55
```

```
rata_rata = tambah(nilai1, nilai2, nilai3)/3  
print(rata_rata)
```

Urutan jalannya program adalah sebagai berikut:

- Baris 1–3 merupakan pendefinisian fungsi **tambah()**, sehingga pada bagian ini program belum menjalankan proses apa pun.
- Baris 5–7 mendefinisikan tiga variabel dan langsung memberikan nilai pada masing-masing variabel tersebut.
- Pada baris 9 fungsi **tambah()** dipanggil dengan tiga argumen yaitu **nilai1 (70)**, **nilai2 (85)**, dan **nilai3 (55)** sehingga program berpindah ke bagian fungsi.
- Parameter **a**, **b**, dan **c** pada fungsi menerima nilai dari argumen tersebut, sehingga **a = 70**, **b = 85**, dan **c = 55**.
- Selanjutnya dibuat variabel **hasil** yang berisi penjumlahan ketiga nilai tersebut, yaitu **70 + 85 + 55 = 210**.
- Nilai **hasil** kemudian dikembalikan menggunakan **return**, yang menandakan fungsi telah selesai dijalankan dan menghasilkan nilai **210**.
- Program kembali ke baris 9 dan nilai tersebut digunakan untuk menghitung rata-rata sehingga diperoleh **rata_rata = 210 / 3 = 70**.
- Pada baris 10 nilai dari variabel **rata_rata**, yaitu **70**, ditampilkan sebagai output.

Optional Argument dan Named Argument

Fungsi dapat memiliki **optional parameter**, yaitu parameter yang bersifat tidak wajib diisi karena sudah memiliki nilai bawaan (**default**) yang telah ditentukan sebelumnya. Untuk membuat optional parameter, nilai default tersebut harus didefinisikan terlebih dahulu saat mendeklarasikan fungsi, seperti yang ditunjukkan pada contoh berikut.

```
def hitung_belanja(belanja, diskon=0):
    bayar = belanja - (belanja * diskon)/100
    return bayar
```

Fungsi **hitung_belanja()** memiliki dua parameter, yaitu **belanja** dan **diskon**. Parameter **diskon** memiliki nilai bawaan (**default**) sebesar 0 yang berarti tidak ada potongan harga. Fungsi **hitung_belanja()** dapat dipanggil dengan beberapa cara seperti pada contoh berikut.

```
def hitung_belanja(belanja, diskon=0):
    bayar = belanja - (belanja * diskon)/100
    return bayar
```

```
print(hitung_belanja(100000))
print(hitung_belanja(100000, 10))
print(hitung_belanja(100000, 50))
```

Output yang dihasilkan adalah sebagai berikut:

```
100000.0
90000.0
50000.0
```

Pemanggilan pertama hanya menggunakan satu argumen, sedangkan pemanggilan kedua dan ketiga menggunakan dua argumen. Setiap parameter dalam fungsi memiliki nama, sehingga saat memanggil fungsi kita juga dapat menyertakan nama parameternya. Dengan cara ini, argumen yang diberikan tidak harus mengikuti urutan parameter yang telah ditentukan. Perhatikan contoh berikut.

```
def cetak(a, b, c):
    print("Nilai a: ",a)
    print("Nilai b: ",b)
    print("Nilai c: ",c)
cetak(b=30, c=40, a=20)
```

Anonymous Function (Lambda)

Sesuai dengan namanya, anonymous function merupakan fungsi yang tidak memiliki nama. Dalam Python, anonymous function merupakan fitur tambahan dan bukan fitur utama. Pada Python, anonymous function didefinisikan menggunakan keyword lambda. Sebagai contoh, perhatikan fungsi source code berikut ini:

```
tambah = lambda a, b: a + b
print(tambah(10,20))
```

Setiap anonymous function pada Python terdiri dari beberapa bagian berikut ini:

- **lambda** digunakan untuk mendefinisikan anonymous function
- Bound variable yaitu argument (angka) pada lambda function
- Body yaitu bagian utama dari lambda, berisi ekspresi yang akan diproses dan menghasilkan suatu nilai

BAGIAN 2: LATIHAN MANDIRI (60%)

Link Github: https://github.com/chinorjodie/71251187_chinor.git

SOAL 1

Source Code:

```
def cek_angka(a,b,c):  
    bedacuy = (a != b) and (a != c) and (b != c)  
    samacuy = (a + b == c) or (a + c == b) or (b + c == a )  
    return bedacuy and samacuy  
  
print(cek_angka(2,7,5))  
print(cek_angka(3,4,8))  
print(cek_angka(1,3,4))  
print(cek_angka(5,2,3))
```

Output:

```
● PS D:\71251187_chinor\minggu05> py .\soal01  
True  
False  
True  
True  
○ PS D:\71251187_chinor\minggu05> █
```

Penjelasan:

Program tersebut mendefinisikan fungsi **cek_angka(a, b, c)** untuk memeriksa kondisi pada tiga buah angka. Di dalam fungsi terdapat variabel **bedacuy** yang mengecek apakah ketiga angka tersebut berbeda satu sama lain, serta variabel **samacuy** yang memeriksa apakah jumlah dua angka sama dengan angka yang lainnya. Fungsi kemudian mengembalikan hasil dari kedua kondisi tersebut menggunakan operator **and**, sehingga hasilnya bernilai **True** jika kedua syarat

terpenuhi. Setelah itu, fungsi dipanggil beberapa kali menggunakan **print()** dengan kombinasi angka yang berbeda, sehingga hasilnya akan ditampilkan di layar dalam bentuk **True** atau **False**.

SOAL 2

Source Code:

```
def digit_belakang(x,y,z):  
    x = x % 10  
    y = y % 10  
    z = z % 10  
    if x == y or x == z or y == z:  
        return True  
    else:  
        return False  
  
a = int(input("Masukkan bilangan pertama: "))  
b = int(input("Masukkan bilangan kedua: "))  
c = int(input("Masukkan bilangan ketiga: "))  
hasil = digit_belakang(a,b,c)  
print ("Hasil:", hasil)
```

Output:

```
● PS D:\71251187_chinor\minggu05> py .\soal2.py  
Masukkan bilangan pertama: 5  
Masukkan bilangan kedua: 5  
Masukkan bilangan ketiga: 5  
Hasil: True  
● PS D:\71251187_chinor\minggu05> py .\soal2.py  
Masukkan bilangan pertama: 6  
Masukkan bilangan kedua: 8  
Masukkan bilangan ketiga: 7  
Hasil: False  
○ PS D:\71251187_chinor\minggu05> █
```

Penjelasan:

Program tersebut mendefinisikan fungsi **digit_belakang(x, y, z)** yang digunakan untuk memeriksa apakah tiga bilangan memiliki **digit terakhir yang sama**. Di dalam fungsi, setiap bilangan diambil digit belakangnya menggunakan operasi **modulus (% 10)**. Setelah itu program mengecek apakah ada dua digit yang sama menggunakan kondisi **x == y, x == z, atau y == z**. Jika ada digit yang sama, fungsi akan mengembalikan nilai **True**, sedangkan jika tidak ada yang sama akan mengembalikan **False**. Selanjutnya program meminta pengguna memasukkan tiga bilangan melalui **input**, menyimpannya ke dalam variabel **a, b, dan c**, lalu memanggil fungsi **digit_belakang()** untuk memeriksa hasilnya. Nilai yang dikembalikan oleh fungsi kemudian disimpan pada variabel **hasil** dan ditampilkan ke layar.

SOAL 3

Source Code:

```
fahrenheit = lambda c: (9/5) * c + 32
reamur = lambda c: 0.8 * c
print(" Input C = 100 -> Output F =", fahrenheit(100))
print(" Input C = 80 -> Output R =", reamur(80))
print(" Input C = 0 -> Output F =", fahrenheit(0))
```

Output:

```
● PS D:\71251187_chinor\minggu05> py .\soal03
  Input C = 100 -> Output F = 212.0
  Input C = 80 -> Output R = 64.0
  Input C = 0 -> Output F = 32.0
○ PS D:\71251187_chinor\minggu05> █
```

Penjelasan:

Program tersebut menggunakan **anonymous function (lambda)** untuk melakukan konversi suhu. Pada baris pertama, dibuat fungsi **fahrenheit** dengan lambda yang menerima parameter **c** (Celcius) dan mengubahnya ke **Fahrenheit** menggunakan rumus $(9/5 \times C + 32)$. Pada baris kedua, dibuat fungsi **reamur** yang juga menerima parameter **c** dan mengubahnya ke **Reamur** dengan rumus $(0.8 \times C)$. Setelah itu, program menampilkan beberapa contoh hasil konversi menggunakan

perintah **print()**, yaitu mengonversi 100°C ke Fahrenheit, 80°C ke Reamur, dan 0°C ke Fahrenheit, lalu hasilnya ditampilkan langsung ke layar.